Solving 3×9 Quoridor with Independently Verifiable Winning-Strategy Certificates

Ulysse Napoly
Independent researcher
Correspondence: ulysse.napoly@gmail.com

***Abstract*—We resolve the two previously open cells of the public Quoridor frontier table: on the 3×9 board, the first player wins with 9 and with 10 walls per player, and with 10 walls the minimax forcing horizon is exactly 35 plies. The upper bound is established by a conservative audit that disables the two specialized pruning modules and proves all 18 reflection classes of the 35 second-player replies in fresh processes (9,309,248,724 nodes); the matching lower bound refutes every first move within 33 plies (3,553,060,577 nodes), and odd-ply parity excludes 34. A structurally different second solver—lazy wall-configuration construction with a union-find legality gate instead of eager enumeration—independently replicates both bounds and the 9-wall result. Beyond replication, we make the winning claim checkable without trusting any solver: a strategy-DAG certificate format (qcert-1) whose 18 parts (191,173,124 state–move pairs) were verified exhaustively by a clean-room verifier sharing no code with the solvers—612,890,536 verification expansions over all 35 replies, zero violations—then accepted end to end by a second independent verifier as a root-position aggregate: 287,795,827 certificate nodes, 749,603,013 regenerated edge obligations, derived bound 35, zero errors. A one-page no-stalemate theorem (height ≥ 3, sharp at height 2) closes the one semantic loophole that could have shifted bounded verdicts. The two halves of the exact-horizon claim rest on evidence of different strength: the 35-ply upper bound is backed by independently checkable strategy certificates, while exactness additionally relies on the replicated exhaustive refutations, which are not yet certificate-backed. We also report exact partial bounds for the next open cell, 4×7 with 7 walls (no forced win for either player within 26 plies), measured move-ordering effects on certified branches, and four audited negative results. All published audits are deterministic and report node counts as reproducibility fingerprints; three representative audit classes reproduced node-for-node on a second machine and compiler.**

***Index Terms*—Quoridor, game solving, bounded adversarial search, strategy certificates, proof verification, replication**

I. Introduction

Solving a game variant exactly—determining its game-theoretic value and a witness strategy—is one of the older exact sciences of artificial intelligence, with a tradition running from Connect Four and Nine Men's Morris through checkers [1]–[4]. Quoridor has long resisted this treatment. A wall-placement race game with perfect information, it combines a large branching factor with path-legality side conditions and, on the full 9×9 board, an intractable configuration space. Determining the winner of generalized $n \times n$ Quoridor was recently shown PSPACE-complete [5], by reduction from Schaefer's positive-CNF game [6], so exact results necessarily target bounded variants.

For narrow Quoridor boards a public frontier table [7] records, per board geometry and wall stock, which side wins. As of this writing it lists nearly all geometries of area at most 28 as solved—with the 3×9 row open at exactly two wall counts, 9 and 10 (Fig. 1). Those two cells are the subject of this paper.

Results. We show that 3×9 Quoridor is a first-player win at both 9 and 10 walls per player, and that at 10 walls the win takes *exactly* 35 plies against best defense (Section V). The witness opening is the central pawn advance. Matching upper- and lower-bound audits, run with the suspect pruning modules disabled and a fresh transposition table per branch, visit 12.9 billion nodes in total; a second solver of deliberately different architecture replicates every verdict.

The verifiability gap. Computational game-solving results are traditionally supported by internal consistency checks and, at best, by independent re-implementation [3], [8], [9]. The strongest evidence practice in automated reasoning goes further: SAT solvers emit machine-checkable proof certificates that an independent—even formally verified—checker validates [10]–[12], a discipline now spreading beyond SAT [13]. We transport that discipline to game solving. Our solver emits a *strategy-DAG certificate*: for every reachable state where the winner is to move, the move to play, with a strictly decreasing certified rank. Checking the certificate requires move generation and ply arithmetic—deterministic enumeration of the certified obligations, with no heuristic game-tree proof search—so a small clean-room program can validate the entire winning claim (Section VI).

Contributions.

- **C0.** First-player wins for $3{\times}9{\times}9$ and $3{\times}9{\times}10$, closing the two open cells of the public frontier, with an exact 35-ply forcing horizon at 10 walls (certificate-backed upper bound; dual-solver replicated exhaustive lower bound).
- **C1.** An independently checkable certificate stack (qcert-1 parts, qcert-aggregate-1 composition): 18 parts verified exhaustively by a clean-room verifier (zero violations over all 35 replies), and an accepted cross-architecture aggregate binding all 18 parts to the initial-position claim.
- **C2.** A no-stalemate theorem for boards of height ≥ 3 with a sharp height-2 counterexample, turning a documented semantics/code divergence into proved dead code for all published claims.
- **C3.** A replication methodology—conservative audits, node-exact cross-toolchain determinism spot checks, differential

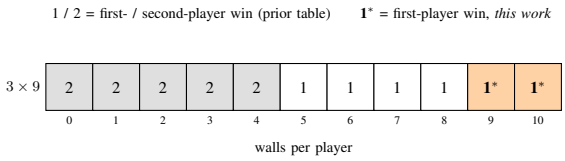

Fig. 1. The 3×9 row of the public Quoridor frontier table [7]. Cells 0–8 were already solved (second-player win through 4 walls, first-player win from 5); the cells at 9 and 10 walls per player were listed as unknown and are closed by this work, at 10 walls with an exact 35-ply horizon.

validation against clean-room rule engines—reusable as an evidence standard for computational game solving.

- **C4.** Exact partial bounds for the next open cell, $4\times7\times7$: neither player forces a win within 26 plies; the central opening forces no first-player win within 27 (Appendix C).
- **C5.** Measured move-ordering effects on certified branches (2–13$\times$ node reductions), a horizon-reversal caution against shallow pilot benchmarks, and four audited negative results (Appendix D).

All claims are backed by archived artifacts (commands, logs, SHA-256-pinned binaries, machine-readable audit records) in the research repositories accompanying this paper; Section VII states precisely what remains trusted. The certificate stack covers the initial position end to end: exhaustively verified per-reply parts, plus an accepted aggregate composition binding all 18 parts to the root claim.

II. Background and Related Work

A. The game

Quoridor is played on a $W \times H$ grid. Each player has a pawn; Player 1 starts at the middle of the bottom row and must reach the top row, Player 2 mirrors. A turn is either a pawn step to an adjacent unblocked cell—with straight jumps over the adjacent opponent and diagonal side-steps when the jump is blocked—or the placement of a two-segment wall from a finite personal stock. A wall may not overlap or cross existing walls and, crucially, may never cut off *either* pawn from its goal row: every placement must preserve a path for both. The path-preservation rule couples wall legality to global connectivity. It is the source of most implementation risk.

We write $W\times H\times k$ for the $W \times H$ board with k walls per player; moves are denoted $P(r,c)$ for pawn moves and $H(r,c)$/$V(r,c)$ for wall anchors. Throughout this paper rows are indexed 0 to $H-1$ starting from Player 1's *goal* side: Player 1 starts at $(H-1,\lfloor W/2\rfloor)$ and wins on row 0, Player 2 mirrors. The rules convention document (`RULES.md`) fixes cell indexing, jump priority, and wall-anchor coordinates; every engine in this paper implements exactly that document, and all coordinates below use this single convention.

B. Prior work on Quoridor

Early academic work built playing agents: Glendenning tuned a linear evaluation with a genetic algorithm [14], Mertens built a rule-based/minimax agent [15], and Monte Carlo tree search agents followed [16]. Exact analysis began with retrograde analysis of reduced boards, solving 5×5 Quoridor with one wall each and related small variants [17], [18]. The broadest exact effort to date is Slatton's frontier campaign [7], which solved nearly all variants of area ≤ 28 for most wall counts with a proof-number-search-related bounded solver [19] over precomputed wall-configuration masks. It published the frontier table this paper extends, and it reports a structural finding we can confirm from the outside: on odd-height boards the winner flips from the second to the first player as wall stocks grow (Fig. 1). Our two new cells continue the first-player regime. The 10-wall horizon of exactly 35 plies quantifies the margin.

C. Evidence standards in game solving

Landmark solved-game results—Connect Four [1], Nine Men's Morris [2], Kalah [20], Awari [8], small Go boards [21], checkers [3], 8×8 Hex [9] (the general game being PSPACE-complete [22], [23]), Gardner minichess [24]—rest on careful internal validation and occasionally independent re-implementation, but none ships an artifact that a third party can check *without* re-running search. The SAT community closed the analogous gap with proof logging: DRAT certificates checked by independent tools [10], scaled to the 200-terabyte Pythagorean-triples proof [11], hardened by formally verified checkers [12], and extended beyond SAT via pseudo-Boolean justifications [13].

Two neighboring communities, moreover, have long produced what are formally strategy certificates. A quantified Boolean formula *is* a two-player game between an existential and a universal player [6], and QBF solvers routinely extract winning strategies as Skolem or Herbrand functions—circuits validated by an external SAT call, independently of the solver that produced them [25], [26], with QRAT proof logging covering even aggressive preprocessing [27]. In constraint programming and combinatorial optimisation, pseudo-Boolean proof logging now certifies symmetry and dominance pruning [28]—the abstract counterparts of the mirror transport and wall-stock dominance table used here.

For board games the natural certificate is the same object—a winning strategy annotated so that its correctness is locally checkable—and Section VI instantiates it for bounded Quoridor proofs. To our knowledge, no prior result in *classical board-game* solving at this scale, a lineage whose landmarks rest on internally validated search, has been delivered with a solver-independent, exhaustively checked strategy certificate. The contribution is thus a bridge rather than an invention: it imports the proof-logging standards of SAT, QBF, and certified optimisation into a domain whose global path-preservation legality resists direct encoding into clauses or pseudo-Boolean constraints without combinatorial blow-up, so the certificate must speak the game's native language of moves and connectivity.

III. Rules Semantics and the Stalemate Theorem

A. Bounded proof semantics

All solvers in this paper decide the bounded predicate $\mathrm{Win}(s, T, d)$: from state s, can player T force a win within d plies? The recursion is existential at T's turns and universal at the opponent's, with terminal wins checked at entry and depth measured in plies (pseudocode in Appendix B). Transposition entries store monotone facts ("T wins within d_1", "T does not win within d_2") [29], [30]; keeping the remaining depth in the state avoids the graph-history-interaction pathology [31]. Because a first-player win can end only on an odd ply, exact horizons follow from an upper bound at d and an exhaustive refutation at $d-2$.

A note on draws and repetitions. The bounded predicate needs no repetition rule and no draw adjudication: $\mathrm{Win}(s, T, d)$ quantifies over *all* continuations of at most d plies, cyclic ones included—a line that revisits a position simply burns budget, which only hurts the attacker. Consequently the published claims are statements about Win, not about any unbounded game value under a particular repetition or draw convention: "first-player win in exactly 35 plies" means $\mathrm{Win}(s_0, P_1, 35)$ holds and $\mathrm{Win}(s_0, P_1, 33)$ fails. Under any full-game semantics in which unbounded play without a win is not a win for Player 1 (draw or loss alike), these bounded facts transfer to "Player 1 wins, and needs 35 plies against best defense"; the semantic boundary is stated here so that no such convention is smuggled in implicitly.

One documented divergence existed between the semantics document and the implementations: the value of a universal node with *zero legal moves* (a stalemate). The mathematical convention (empty conjunction) makes it a win for T; the code returns `false` for both targets. If stalemates were reachable, bounded verdicts could depend on the convention. We close this loophole with a theorem rather than an assumption.

B. The no-stalemate theorem

Call a state *legal* if the pawns occupy distinct cells and each pawn's cell is wall-connected to some cell of its own goal row—the exact enumeration domain of the exhaustive audits, a strict superset of the reachable states (on $4{\times}3{\times}3$: 63,184 legal non-terminal states versus 62,732 reachable).

Lemma 1 (path invariant)**.** *In every reachable state, each pawn is wall-connected to its goal row (walls block edges; pawns do not).*

Theorem 1 (no stalemate, $H \geq 3$)**.** *On any board with $H \geq 3$, any width and any wall stocks, every legal non-terminal state grants the player to move at least one pawn move.*

Proof sketch. Suppose the mover A at cell p has no pawn move. Since the state is non-terminal and legal, p's component contains a goal-row cell other than p, so p has an unblocked neighbor; if that neighbor were empty it would be a legal step, so p's *only* unblocked neighbor is the opponent's cell n. The failed jump and both failed diagonals then force every edge at n other than n–p to be blocked or out of bounds, so the component of p is exactly $\{p, n\}$. Applying legality *twice*—once per pawn—this two-cell prison must contain a cell of row 0 and a cell of row $H-1$; adjacency gives $H \leq 2$. □

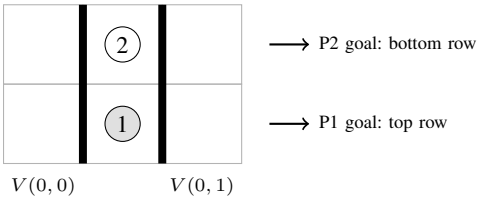

Fig. 2. Sharpness at $H = 2$: on the 3×2 board, $V(0,0)$ then $V(0,1)$ (thick segments) seal the central column. Player 1 (to move) has no pawn move—the only unblocked neighbor is occupied, the jump lands out of bounds, both diagonals are blocked—and both wall anchors are used, so no move exists at all, even with walls in stock. Both pawns still satisfy the legality invariant, sitting on each other's goal rows: exactly the two-cell prison the theorem's final step excludes for $H \geq 3$.

TABLE I
Mechanical verification of Theorem 1 (clean-room engine, archived scans).

Scan family	States	Pawn-move-less
Reachable, 15 variants, $H \geq 3$	≈1.9M	0
Legal superset, 8 variants, $H \geq 3$	≈3.4M	0
Adversarial hunt, 9 topologies	≈17.8M	0
$H = 2$ negative controls	—	found at ply 2

Two features of the proof matter. First, it needs *both* pawns' connectivity: the mover-only argument ("the first vertex of the mover's shortest path is free, jumpable, or bypassable") is irreparable, because the mover's path may legitimately run through the opponent's cell. Second, it never uses the geometric wall-placement rules, only connectivity, so it covers the audits' enumeration domain a fortiori. The full proof, including the jump/diagonal edge case analysis, is given in Appendix A.

C. Sharpness, mechanical verification, consequences

The bound is sharp: Fig. 2 shows a stalemate reachable in two plies on the 3×2 board. On height-2 boards the convention is semantically live—exhaustive dual-convention solves measure 7, 12, and 42 diverging state verdicts on $3{\times}2{\times}1$, $3{\times}2{\times}2$, and $4{\times}2{\times}2$ respectively—so height-2 claims must fix the convention explicitly. For $H \geq 3$ the divergent branch is dead code.

The theorem was verified mechanically by a clean-room engine (Table I): forward scans of all reachable states, direct enumerations of the legal superset (independently reproducing the archived 63,184 count on $4{\times}3{\times}3$), and an adversarial hunt over degenerate topologies with a pawn-move generator re-implemented from the rules document alone. Every $H \geq 3$ scan found zero pawn-move-less states; $H = 2$ controls found stalemates exactly as constructed. Every production audit additionally archives a `stalemates_seen` counter, pinned to zero as a runtime tripwire.

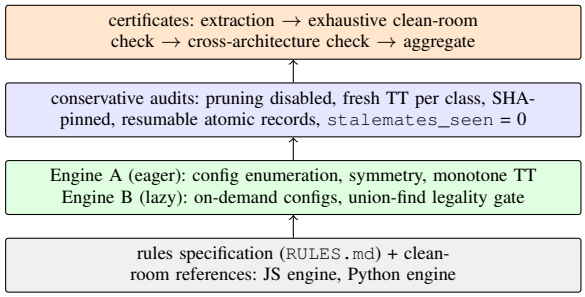

Fig. 3. The evidence stack. Differential grids tie every layer back to the rules specification; the certificate layer at the top is checkable without trusting anything below it except the rules reading and ply arithmetic.

TABLE II
DIFFERENTIAL VALIDATION INVENTORY (ALL ARCHIVED; ZERO DIVERGENCE THROUGHOUT).

Check	Scale
Exhaustive legal-move grid, $4\times3\times3$	63,184 states
Random move grids, $3\times9\times10$ / $4\times7\times7$	2,000 / 1,500 states
Exhaustive bounded-proof grid, $3\times3\times1$	51,408 queries
Sampled proof grid, $4\times3\times2$	5,000 queries
Known-variant regressions (both winners)	6 variants
Wall universes, independent recount	2 variants
Root/reply reflection partitions	re-derived
Sanitizers (ASan, UBSan), Clang rebuilds	tractable suites
Union-find gate vs. BFS (Engine B)	121,418 verdicts

IV. SOLVERS AND AUDIT METHODOLOGY

Two exact solvers of deliberately different architecture, three clean-room rule implementations, and a layered audit protocol produce and check every claim (Fig. 3).

A. Engine A: eager enumeration

The primary solver pre-enumerates all legal wall configurations of the variant (2,929,319 for $3\times9\times10$; 16,368,423 for $4\times7\times7$) and stores, per configuration, goal reachability, distances, movement edges, legal wall additions, and symmetry transforms. Search is the bounded existential/universal recursion—AND/OR proof search in the tradition of [19], [32]—with iterative deepening, horizontal-mirror and 180° player-swap symmetry, depth-parity normalization, and a two-slot transposition table with full-key equality. Move ordering affects performance only; verdicts are ordering-invariant, a property the portfolio harness enforces by construction (Appendix D).

B. Engine B: lazy replication solver

The replication solver was written against the rules and semantics documents with a deliberately different design. It never enumerates the configuration universe, building wall configurations only when the current proof obligation reaches them. Wall legality is decided by an $O(\alpha)$ union-find cycle gate over the junction lattice: a wall can disconnect a pawn only by closing a cycle with existing walls or the border, each wall contributes two half-segments, and a placement is safe when no cycle closes; BFS runs only in the rare cycle-closing cases. A built-in self-test compares the gate against BFS on 121,418 legality verdicts with zero divergence. Profiling confirms the design premise: a single proof branch touches 0.3–4.6% of the configuration universe.

C. Audit protocol

Every published bound comes from a *conservative audit*, extending the internal-validation practice of earlier solved-game efforts [3], [8]: the two specialized pruning modules most deserving suspicion—the admissible distance bound and the pawn-only endgame table—are disabled; each decomposed class runs in a fresh process with a fresh transposition table; binaries are SHA-256-pinned; every record is written atomically and the harness resumes idempotently; and each record archives node counts, depths, timings, and the theorem-backed `stalemates_seen`=0 counter. Class manifests are re-parsed from the binary and hash-bound, so an ordering change cannot silently move a historical representative.

D. Differential validation

Table II summarizes the validation grids. The two reference engines were independently implemented from the frozen rules and proof-semantics specifications, without consulting or reusing production-solver source code: the JS engine was written from `RULES.md` and `PROOF_SEMANTICS.md` alone, without reading the C++ or Python sources, and independently re-derives the wall-configuration counts by transfer-matrix dynamic programming. Six fully solved small variants match the public table [7], a fourth independent source for those values.

E. Measurement conventions and environments

Throughout the paper, one *node* is one entry into the bounded recursion $\mathrm{Win}(s, T, d)$—counted at function entry, before terminal, transposition, and pruning checks resolve the call—in both engines. The two engines' node counts are *not* directly comparable: they differ in symmetry normalization, transposition policy, and move ordering, so cross-engine ratios measure architecture, not difficulty. For the verifiers, one *expansion* (exhaustive C++ verifier) is one state whose local obligations are actually checked—states already verified at an equal or smaller remaining budget are memoized and not recounted—and one *regenerated edge obligation* (streaming aggregate verifier) is one legal move regenerated from the independent rules engine and checked against the certificate's local obligations.

Principal environments, archived with the artifacts: the Engine A conservative audits ran in a Debian container (g++ 14.2.0, `-O3 -DNDEBUG -std=c++20 -march=native`) on an AMD EPYC 9V74 slice (5 vCPU, 4 GiB cgroup memory limit); Engine B production runs, the certificate pipeline, and the aggregate verification ran on a 12-core Windows 11 machine under MSYS2 g++ 16.1; the cross-toolchain reproduction machine ran Ubuntu 24.04 with g++ 13.3. The frozen specification documents are pinned by SHA-256 in the bundle

manifest: `RULES.md` at `0f9a91d1a126fbd2...` and `PROOF_SEMANTICS.md` at `7f70c21e4f861edd...` (full hashes in `MANIFEST.sha256` at the archived commit).

F. Determinism as a reproducibility fingerprint

The search is deterministic and portable: re-running archived audit commands on a different machine, compiler, and freshly compiled binary reproduces node counts *exactly* (three classes checked: 79,766,979, 142,908,257, and 44,048,383 nodes). We report node counts throughout as fingerprints that any replicator can match bit-for-bit; the claims themselves rest on verdicts, not counts.

V. Results: The 3×9 Frontier Cells

A. 3×9, 10 walls: a first-player win in exactly 35 plies

After the central opening $P(7,1)$ (the pawn advance $(8,1) \to (7,1)$), Player 2 has 35 legal replies, which horizontal reflection partitions into 18 exact classes; the partition is re-derived by independent scripts. The conservative audit proves every representative in a fresh process. That gives the 35-ply upper bound from the initial position. The lower-bound audit fixes each of the 18 reflection classes of Player 1's first moves and exhaustively refutes a win within the remaining 32 plies—no proof, no timeout—so no first-player win exists within 33 plies. Wins for Player 1 occur only on odd plies, excluding 34. Hence the minimax forcing horizon is exactly 35. The two halves of this statement carry evidence of different strength: the upper bound is additionally certificate-backed (Section VI), while the 33-ply refutations rest on the dual-solver exhaustive audits. Table III gives the numbers for both engines. The two architectures agree on all 36 verdicts and on the fine structure of the proof: 17 classes prove at remaining depth 31 and the retreat reply $P(1,1)$ alone requires 33, in both engines and at both wall counts.

B. 3×9, 9 walls

The same protocol proves the first-player win at 9 walls (upper bound 35), covering the second open cell. The public table records winners, not horizons [7]; our lower-bound replication at 10 walls completes the exact-horizon statement where it is claimed.

C. What replication adds

Engine B shares no search code, no state representation, no configuration enumeration, and no wall-legality algorithm with Engine A. Its agreement on all 54 decomposed class verdicts across the two variants—36 upper-bound classes and 18 lower-bound refutations—makes a correlated *search* defect implausible; the remaining single point of failure—a shared misreading of the rules—is addressed by the clean-room differential grids of Table II and, more strongly, by the certificates of the next section.

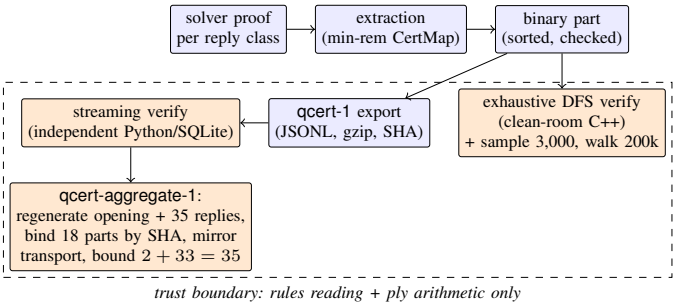

Fig. 4. Certificate pipeline. Emission (blue) may use solver data structures; every verifier (orange) is independent of the solvers—the exhaustive C++ verifier shares zero code with them, and the streaming verifier was written by the other research line against the format specification.

VI. Independently Verifiable Certificates

Replication still asks a skeptic to trust two searches. The certificate layer removes that trust: it packages the winning strategy itself in a form checkable by deterministic exhaustive enumeration of the certified obligations—no heuristic game-tree proof search, no solver code (Fig. 4).

A. Format

A qcert-1 certificate is a JSONL strategy DAG for a claim "player T wins from root r within d plies." Each line is one deduplicated state with its certified rank; at states where T moves it declares the strategy move. The verifier's obligations are purely local: the root is present at rank $\leq d$; at every T-node the declared move is legal and leads to a node of strictly smaller rank (or an immediate terminal win); at every opponent node *all* legal replies are covered by nodes of strictly smaller rank; terminal states are wins for T. Rank strict decrease makes well-foundedness a syntactic property—no cycle can certify—and the ply bound becomes arithmetic on ranks (the obligations are listed as inference rules in Appendix B). The format stores certified ranks rather than exact distances-to-mate: any consistent decreasing assignment is sound, which keeps emission cheap. A hardened lexical layer (UTF-8 without BOM, safe-integer grammar, gzip integrity binding) is shared by all verifiers; adversarial corpora of 10, 20, and 33 corruption families are rejected by the in-memory, streaming, and aggregate verifiers respectively. The local-obligation design deliberately mirrors clausal proof checking in SAT [10], whose certificates have scaled to hundreds of terabytes [11].

B. Extraction and the min-rem invariant

Parts are extracted per reply class from the prover's memo. One extraction bug became a design rule. When the same state is certified twice—once deep inside a proof with a tight remaining budget, once shallowly with a loose one—keeping the *larger* budget's entry can create rank plateaus that let the merged strategy cycle between certified states (observed as a two-state loop between $P(3,2)$ and $P(4,2)$). The fix is a single invariant: the certificate map only ever keeps the *minimum* remaining-ply certification per state. This

TABLE III

THE 3×9×10 BOUNDS AND THEIR INDEPENDENT REPLICATION. UPPER AUDITS PROVE ALL 18 REFLECTION CLASSES OF THE 35 REPLIES TO THE WITNESS OPENING; LOWER AUDITS REFUTE ALL 18 FIRST-MOVE CLASSES WITHIN 33 PLIES. ENGINE A AUDITS DISABLE THE DISTANCE BOUND AND PAWN-ONLY TABLE WITH A FRESH TT PER CLASS.

Bound	Solver	Classes	Nodes	Search time (s)
Upper: win ≤ 35	Engine A (conservative audit)	18/18	9,309,248,724	1,805.6
Upper: win ≤ 35	Engine B (per-class TT)	18/18	5,402,042,173	—
Lower: no win ≤ 33	Engine A (conservative audit)	18/18	3,553,060,577	716.5
Lower: no win ≤ 33	Engine B	18/18	2,505,091,973	1,171
3×9×9 upper: win ≤ 35	Engine A (monolithic + 18-class)	18/18	1,909,467,037[a]	452.3
3×9×9 upper: win ≤ 35	Engine B	18/18	6,577,365,458	—

[a] Monolithic run; the decomposed 18-class verification is archived separately (slowest class 2,514,031,626 nodes).

min-rem rule restores strict decrease by construction, and a forensic reproducer for the pre-fix behavior is archived. For the same reason we do *not* claim the merged single-file strategy: min-per-key merging across parts mixes tempi, with rare rank violations measured at roughly one per 900 states. The deliverable is the 18 per-class parts, which are individually self-contained.

C. Verification results

Table IV summarizes. The 18 parts cover all 35 legal replies: 18 canonical representatives verified directly and 17 mirror twins verified by applying the reflection automorphism from ply 1 (an earlier intra-part mirror fallback was found unsound and removed). The exhaustive pass uses a clean-room C++ verifier—a port of the Python reference checker sharing zero code with either solver—and performs 612,890,536 verification expansions across the 35 branches with zero violations, in 1,262.6 s. Mirror-twin pairs exhibit identical memo cardinalities, an isomorphism consistency check that comes for free.

Cross-architecture validation bridges the two research lines. The $H(1,0)$ part was exported to qcert-1 (881,735,925 bytes, SHA-256-pinned; the export is bit-reproducible from the memo) and accepted by the *other* line's independently written streaming verifier: 9,836,857 nodes, 25,175,363 regenerated edges, root budget 31, zero errors, 1,239 s. That verifier regenerates every move obligation from the independent Python rules engine against a disk-backed index. A further differential drew 1,500 wall configurations from the accepted index and compared the optimized connectivity profiles against naive BFS: 10,024 legal moves and 706,806 all-cell goal-connectivity checks, zero divergence.

D. Composing parts into the root claim

Eighteen branch certificates do not yet formally certify the initial position; the missing step is bookkeeping, and it too is specified and independently checkable. A qcert-aggregate-1 manifest regenerates the witness opening and the exact set of 35 replies, binds each reply orbit to its part by stored-byte SHA-256, transports mirror-twin strategies through the documented reflection automorphism (one reply is self-mirror), verifies each part with a fresh index, and derives the bound $2 + 33 = 35$ explicitly. The aggregate verifier rejects 33 corruption families, including path aliasing and manifest tampering, and is validated end to end on a small complete fixture.

TABLE IV

CERTIFICATE VERIFICATION FOR 3×9×10 (WITNESS OPENING $P(7,1)$; 18 PARTS, 191,173,124 STATE–MOVE PAIRS).

Check	Scale	Viol.
Sampled adversary games (per part)	$18 \times 3{,}000$	0
Bounded DFS walks (per part)	$18 \times 200{,}000$	0
Exhaustive clean-room pass	35/35; 612.9M expansions	0
Cross-arch. acceptance of $H(1,0)$	9.84M nodes, 25.2M edges	0
Profile differential on $H(1,0)$	706,806 checks	0
18-part aggregate acceptance	287.8M nodes, 749.6M edges	0

The production aggregate over the 18 real parts has been accepted. Every part passed under both the identity and mirror-columns transforms—287,795,827 certificate nodes in total, exactly matching the extraction-memo state count, with 749,603,013 regenerated edge obligations—the 35 replies were regenerated and matched to their 18 orbits, and the bound was derived explicitly as $2 + 33 = 35$, in 30,232 s under the SHA-frozen streaming verifier (Table IV, last row). The initial-position claim is therefore certificate-backed end to end. One bookkeeping note: the aggregate's $H(0,0)$ part is a regenerated, equally verified strategy, distinct from the archived binary part used in the exhaustive pass; the two are valid alternatives whose cardinalities must not be summed.

E. Trust boundary

What must a skeptic still trust? (i) that `RULES.md` captures the intended game—checked socially, and against the public table on six solved variants; (ii) the verifier they choose to run—each is small, search-free, and available in three independent implementations (in-memory JS, streaming Python/SQLite, exhaustive C++), written by different parties, two of them clean-room; and (iii) ply arithmetic. Notably *excluded* from the trusted base: both solvers, the transposition logic, the symmetry code, the pruning modules, and the extraction pipeline. The lower bound (no win within 33) is the one claim not yet certificate-backed; it rests on the dual-solver exhaustive refutations, and Section VII sketches survival certificates as the designed remedy.

VII. Limitations and Future Work

Not a machine-checked proof. The result is a certificate-checked computational claim, not a formalized theorem. The trusted base (Section VI) is small but informal: a rules document read by humans and four independent implementations. Formalizing qcert-1 checking in a proof assistant, following the SAT community's path [12], is the natural next step and the format was designed to make it cheap (local obligations, syntactic well-foundedness).

The lower bound is uncertified. "No win within 33" rests on replicated exhaustive refutations by two solvers. The designed remedy is a *survival certificate*: at every defender node, a move keeping the attacker from winning for $\geq k$ further plies, with the verifier enumerating all attacker moves—the same machinery with the quantifiers exchanged. Its cost is unmeasured (the DAG branches on the attacker's side); small-variant estimation comes first.

Shared lineage. "Independence" in this paper decomposes as follows: the two solvers share no code, no state representation, and (mostly) no algorithms, but they do share the frozen specification documents and a single coordinated research process; they are independent implementations, not independent institutional replications. The verifiers share no code with either solver, and two of them were written from the specifications alone; all components nonetheless share the same rules reading. The clean-room engines mitigate but do not eliminate specification risk. The strongest external check available—agreement with the public table [7] on six fully solved variants and on the qualitative wall-count structure of the 3×9 row—is consistent throughout.

Open frontier. $4{\times}7{\times}7$ remains open: 38 first moves await refutation at ply 27 (or a win witness), with the manifest prepared and exact partial bounds already in place (Appendix C). Beyond it lie the remaining $4{\times}7$ stocks and the $7{\times}5$ geometry. On the methods side, the certified corpora themselves are new research material: 287.8 million certified states annotated with winning moves and strictly decreasing upper-bound ranks (certified moves need not be depth-optimal, and ranks are consistent bounds, not exact distances to mate) form a supervision set for learned ordering selectors; a small feasibility probe on this corpus is archived with the artifacts, and a proper study—dataset splits, leakage analysis, and full-depth regret evaluation against exact portfolios—is future work. One small theory item still awaits a clean write-up: the wall-stock Pareto dominance lemma already used by the lazy engine.

Artifact availability. The complete artifact bundle (solver, replication line, certificate pipeline, verifiers, audit records, proof journals) is published at https://github.com/napolocreed/quoridor_3x9 and archived at DOI `10.5281/zenodo.21763852`. The repository tree is byte-pinned by a SHA-256 manifest checked in CI and by a top-level `verify_bundle.py` entry point; the multi-gigabyte certificate parts ship as release assets whose SHA-256 hashes are pinned inside the tree (certificate parts compress to practical sizes, gzip $\approx 7\times$ on JSONL exports). The frontier-table maintainer has been contacted regarding the review format for updating the public record.

VIII. Conclusion

The two open 3×9 cells of the public Quoridor frontier are closed: first-player wins at 9 and 10 walls per player, the latter in exactly 35 plies. The 35-ply upper bound is backed by independently checkable strategy certificates; exactness additionally relies on replicated exhaustive refutations showing that no first-player win exists within 33 plies. The result comes with an unusually extensive evidence package for a classical board-game solving claim: conservative audits, deterministic node-count fingerprints reproduced across toolchains on representative classes, full replication by a structurally different solver, a theorem eliminating the one semantic loophole, and—new to this setting—a solver-independent strategy certificate, verified exhaustively per reply and accepted end to end for the initial position by a second independent verifier. Certificates change the economics of belief in such results: checking is cheaper than solving, delegable, and robust to every defect class that plagues search code. We expect the practice to transfer well beyond Quoridor.

Acknowledgments and Disclosure

The computational research reported here was carried out by two AI systems working as coordinated but mutually auditing agents—GPT 5.6 ("sol"), which built the primary solver and audit apparatus, and Claude (Anthropic), which built the replication solver, the certificate pipeline, and the clean-room verifiers—under the direction of the human author, on consumer hardware. The project began as the author's informal comparison of the two systems on the same research question; when their independent results outgrew the author's ability to adjudicate between them, the comparison was restructured into the mutually auditing protocol this paper describes. Cross-review documents, coordination logs, and both codebases are part of the published artifacts. The human author assumes sole responsibility for the frozen rules specification, the stated theorems, the selection of experiments, the published artifacts, and the final scientific claims. The author thanks Grant Slatton for maintaining the public frontier table that motivated this work.

Appendix A
Full Proof of the No-Stalemate Theorem

This appendix gives the complete proofs of the lemma and theorem of Section III, following the archived theorem document (`emitter/STALEMATE_THEOREM.md`) verbatim in substance. Wall-connectivity is the graph on cells whose edges are the orthogonally adjacent pairs not blocked by a wall; pawns never block edges of this graph (the rules define wall legality on unblocked cell edges; pawns are not walls, a pawn-transparent reading adopted identically by every engine in this paper).

Proof of the path invariant (Lemma 1). By induction over the move sequence. *Base:* the initial position has no walls; the

board is one component containing all rows. *Wall placement:* legality explicitly requires that, after placement, each pawn has a path from its current cell to its own goal row—precisely the invariant. *Pawn move:* walls are unchanged, so components are unchanged; it suffices to show the moving pawn stays in its component. A *step* traverses an unblocked edge, so origin and destination are in one component. A *straight jump* over the opponent at n to z requires the edges p–n and n–z unblocked, so p and z are connected through n. A *diagonal* to a perpendicular neighbor z of n requires the approach edge p–n and the connecting edge n–z unblocked, so again p and z are connected through n. The non-moving pawn does not move. □

Two reading notes. (a) The jump and diagonal cases assume the approach edge p–n is unblocked; the rules' special-move clause ("when the opposing pawn occupies that adjacent cell") refers to a cell reachable over an unblocked shared edge, and every implementation reads it that way. Under the alternative reading (special moves across a blocked approach edge) the theorem below survives a fortiori—more available moves only helps the mover—but the lemma's diagonal case needs the presupposition. (b) With two pawns, jump and diagonal destinations are never occupied: the only other pawn sits at p, in the approach direction.

Proof of Theorem 1 (no stalemate, $H \geq 3$). Call a state *legal* when the two pawns occupy distinct cells and each pawn's cell lies in the same wall-connectivity component as a cell of its goal row—the exact enumeration domain of the exhaustive audits, with no reachability requirement; by the lemma, every reachable state is legal. Suppose for contradiction that in some legal non-terminal state the player to move—call them A, pawn at p, goal row g_A—has no legal pawn move. Write B for the opponent, pawn at n, goal row g_B; $\{g_A, g_B\} = \{0, H-1\}$.

Step 1: p has at least one unblocked neighbor. The state is non-terminal, so p is not on row g_A. By legality, p's component contains a cell of row g_A, hence a cell other than p, hence the component has at least two cells and p has an unblocked edge.

Step 2: every unblocked neighbor of p is occupied by B. An empty unblocked neighbor is always a legal step. There is only one opponent, so p has exactly one unblocked neighbor, namely B's cell n.

Step 3: n's only unblocked edge is n–p. Since A has no legal move: the straight jump fails, so the cell behind n (in direction $p \to n$) is out of bounds or its edge from n is blocked; by the rules, diagonals are then available, and both fail, so each perpendicular neighbor of n is out of bounds or its edge from n is blocked. The four potential edges at n are: toward p (unblocked, by Step 2), straight behind, and the two perpendiculars. Hence n's unblocked edges are exactly $\{n$–$p\}$.

Step 4: the component of p is exactly $\{p, n\}$. By Steps 2 and 3.

Step 5: contradiction. Applying legality to A: row g_A intersects $\{p, n\}$; p is not on g_A (non-terminal), so $\mathrm{row}(n) = g_A$. Applying legality to B: row g_B intersects $\{p, n\}$; n is not on g_B (non-terminal), so $\mathrm{row}(p) = g_B$. Thus $\{\mathrm{row}(p), \mathrm{row}(n)\} = \{0, H-1\}$. But p and n are orthogonally adjacent, so $|\mathrm{row}(p) - \mathrm{row}(n)| \leq 1$, forcing $H - 1 \leq 1$, i.e. $H \leq 2$: contradiction with $H \geq 3$. □

The theorem needs neither reachability nor stock consistency: it holds on the strict legal superset that the exhaustive audits enumerate (on $4\times3\times3$: 63,184 legal non-terminal states versus 62,732 reachable—452 legal states are unreachable). It is also strictly stronger than "no stalemate": it guarantees a *pawn* move even when the mover has walls in stock, so `stalemates_seen` $= 0$ is a theorem for $H \geq 3$, not an empirical observation—though the counter stays in every audit as an implementation tripwire. Finally, the proof never uses the geometric wall-placement rules (no-cross, no-overlap), only connectivity, so it holds even for wall bitmasks that violate placement geometry but keep both pawns goal-connected. Sharpness at $H = 2$ is witnessed by the reachable stalemate of Fig. 2, which realizes exactly the two-cell prison of Step 4 with $\mathrm{row}(n) = g_A$ and $\mathrm{row}(p) = g_B$.

APPENDIX B
BOUNDED RECURSION AND CERTIFICATE OBLIGATIONS

A. The bounded predicate

Both engines decide $\mathrm{Win}(s, T, d)$ by the same recursion; they differ only in state representation, wall-legality decision, symmetry normalization, and transposition policy.

1) If s is terminal, return $[\mathrm{winner}(s) = T]$.
2) If $d = 0$, return **false**.
3) If the transposition table holds a monotone fact deciding the query—"T wins within d_1" with $d_1 \leq d$, or "T does not win within d_2" with $d_2 \geq d$—return it.
4) Let $L(s)$ be the legal moves of s ($L(s) \neq \emptyset$ for pawn moves by Theorem 1, so the empty-conjunction branch is dead code for $H \geq 3$).
5) If the player to move is T: result $\leftarrow \bigvee_{m \in L(s)} \mathrm{Win}(\mathrm{apply}(s, m), T, d - 1)$; otherwise result $\leftarrow \bigwedge_{m \in L(s)} \mathrm{Win}(\mathrm{apply}(s, m), T, d - 1)$.
6) Store the monotone fact implied by the result at depth d; return it.

One *node* is one entry into this recursion (Section IV-E). Symmetry normalization maps s to a canonical representative before steps 3 and 6; the conservative audits disable the admissible distance bound and the pawn-only endgame table, so the recursion above is exactly what they execute.

B. *qcert-1* verification obligations

A certificate for "player T wins from root r within d plies" is a set of records $(s, rank_s, move_s)$, with $move_s$ present exactly when T moves at s. The verifier accepts iff all of the following hold; every obligation is local to one record and its regenerated successors:

1) *Root:* r is present with $rank_r \leq d$.
2) *T-node:* $move_s$ is legal in s (regenerated from the rules engine), and $\mathrm{apply}(s, move_s)$ is either an immediate terminal win for T or present with strictly smaller rank.

TABLE V
EXACT PARTIAL BOUNDS FOR 4×7×7 (GAME VALUE OPEN).

Audit	Branches	Nodes
P1 root refutation ≤ 25 plies	39/39	4,888,712,130
P2 refutation ≤ 26 plies	complete	901,837,164
Central opening refuted ≤ 27	40/40	3,591,324,070

3) *Opponent node: every* legal reply $m \in L(s)$ leads to a state that is an immediate terminal win for T or present with strictly smaller rank.
4) *Terminal:* every terminal state in the certificate is a win for T.
5) *Well-foundedness:* ranks are nonnegative integers, so strict decrease along every certified line is a syntactic property—no cycle can certify—and the ply bound is arithmetic on ranks.

Soundness is immediate by induction on $rank_s$: at every certified state, T either wins immediately or forces play into certified states of strictly smaller rank, hence wins within $rank_s$ plies. The verifier never searches: it enumerates each record's obligations once, in any order.

APPENDIX C
FRONTIER PROGRESS ON 4×7 QUORIDOR

The next open cell in the same table [7] is 4×7 with 7 walls per player. It remains open, but exact partial bounds now constrain it (Table V): fixing each of Player 1's 39 legal first moves and refuting a win in the remaining 24 plies shows no first-player win within 25 plies; a completed Player-2 audit through depth 26 shows no second-player win within 26; and after the central exchange $P(5,2)$, $P(1,2)$, all 40 third moves are refuted at the remaining 24-ply bound, so the central opening forces no win within 27 plies. All audits use the conservative protocol (pruning disabled, fresh 64-bit-keyed tables). A memory redesign (deriving the opponent's distance tables by exact 180° rotation instead of storing both) cut precompute RSS from 2.95 to 2.15 GiB and, reproducing archived branches node-exactly, enables larger transposition tables for the coming campaigns.

APPENDIX D
ORDERING PORTFOLIOS, RACE ENDGAMES, AND NEGATIVE RESULTS

With certified branches available, move-ordering policies can be raced under a hard soundness constraint: same named branch, same verdict, fresh tables. Two findings and four audited negatives came out of this harness (Table VI).

Path-flow ordering. Scoring candidate walls by their impact on the opponent's whole shortest-path DAG (how many shortest routes an edge carries) is a strong exact ordering signal in corridor positions: on the archived $P(4,2)$ branch of $4\times7\times7$ it cuts the proof from 119.9M to 47.2M nodes, and on two certified $3\times9\times10$ replies it roughly halves nodes and wall time. In the lazy engine, a related route-choice count ordering cuts a hard branch from 223.2M nodes (47.3 s) to 15.3M (3.7 s), and combining it with an exact wall-stock dominance table (a Pareto argument: extra own walls never hurt, extra opponent walls never help) reaches 12.9M nodes.

Horizon reversal. The same harness produced a caution: on $P(4,2)$, the policy that wins at shallow pilot depths *loses* at full depth. Policy selection from shallow pilots is unsound in this domain; portfolios must race at the target horizon with stable named moves (an earlier negative diagnosis of this very signal was itself invalidated because index-based move naming let the ordering change the tested position).

Exact race endgames. Once wall stocks are exhausted, Quoridor reduces to a pure pawn race, and this subgame turns out to admit a complete, exactly verified characterization. Sweeping the full fixed-point race tables of 24 boards ($W \in \{2,3,5,9\}$, $H \in \{4,\ldots,9\}$; the 9×9 table self-checks against an independently validated implementation on all 10,242 states) establishes a dichotomy: with equal goal distances d, the player to move *loses*—in exactly $2d$ plies—if and only if the pawns face each other on the same column with an even row gap (possible only on odd-height boards, where the initial position is the extreme case), and *wins* in every other equal-distance configuration; no drawn race state exists on any swept board. The mechanism is tempo: a blocked face-to-face jump hands over exactly one tempo, so column alignment at even gap is pure zugzwang. The dichotomy is established computationally and backed by a complete imitation-strategy proof in the artifacts (a jump-tempo lemma, a contact-parity invariant, and an interception lemma); the proof was then attacked by an independent adversarial audit, which pierced three minor gaps—including a genuine accounting error omitting backward jumps—that were repaired with the audit trail preserved in the same document. Practically, the characterization is a three-comparison exact oracle at search leaves with exhausted stocks, and a caution that race parity *inverts* relative to walled play, where the initiative wins.

Certificates as refutation oracles. The certificate corpus also pays back into search. The upper-bound CertMaps are monotone facts ("the attacker wins from s within r"), so a refutation search may consult them read-only: at any node whose normalized budget is at least the certified rank, the win-exists subproof collapses without expansion; a mirrored-key probe extends coverage through the documented reflection automorphism. Merging the 18 per-reply maps deduplicates 287.8M certified states to 162.6M unique ones—43% of certified states are shared across reply branches through wall-attribution transpositions—and a single open-addressed table keeps the probe near one cache miss (a first variant probing 16 per-reply tables sequentially lost its node gains to memory traffic, 2.4× wall time). Replaying the replication solver's 18 first-move refutations with this oracle preserves all 18 verdicts and archived baselines exactly, and cuts 19.9% of nodes (2,505,091,973 to 2,006,736,746) and 34.4% of wall time (1,171 s to 768 s), every class gaining; the central-pawn refutation, the largest, drops 31.3% of nodes and 44.9% of time. Without seeds the modified binary reproduces the archived runs node-exactly. The same protocol transfers to the

TABLE VI
ORDERING RACES ON BRANCHES WITH CERTIFIED OUTCOMES (VERDICTS PRESERVED IN EVERY RACE).

Branch (policy)	Nodes	Time (s)
$4{\times}7{\times}7$ $P(4,2)$ baseline	119,898,717	—
+ path-flow	47,214,748	—
$3{\times}9{\times}10$ $P(7,1)\,V(1,0)$ baseline	79,800,528	16.0
+ path-flow	37,279,448	9.5
$3{\times}9{\times}10$ $P(7,1)\,V(5,0)$ baseline	142,948,371	29.9
+ path-flow	50,633,369	13.2
Lazy $4{\times}7{\times}7$ $H(5,2)$ baseline	223,188,594	47.3
+ route-choice ordering	15,323,311	3.7
+ Pareto stock table	12,875,101	3.3

$4{\times}7{\times}7$ campaign: retain the upper-bound maps per branch and seed the 38 refutations at ply 27.

Negative results, kept. Four redesigns failed under exact benchmarks and are documented with regression tests rather than discarded: a correct bounded DF-PN prototype [19], [33] remains slower than the mature DFS solver on frontier branches; decremental shortest-distance repair tripled runtime at identical node counts; and both the naive-DFS deferral and the cache-first local-touch variants of the union-find legality gate lost to their baseline after independent audit fixed two defects in the experimental build.

REFERENCES

[1] L. V. Allis, "A knowledge-based approach of Connect-Four," M.Sc. thesis, Vrije Universiteit Amsterdam, 1988.

[2] R. Gasser, "Solving nine men's morris," *Computational Intelligence*, vol. 12, no. 1, pp. 24–41, 1996.

[3] J. Schaeffer, N. Burch, Y. Björnsson, A. Kishimoto, M. Müller, R. Lake, P. Lu, and S. Sutphen, "Checkers is solved," *Science*, vol. 317, no. 5844, pp. 1518–1522, 2007.

[4] H. J. van den Herik, J. W. H. M. Uiterwijk, and J. van Rijswijck, "Games solved: Now and in the future," *Artificial Intelligence*, vol. 134, no. 1–2, pp. 277–311, 2002.

[5] M. Drop, B. G. Rin, and F. van der Velde, "Quoridor is PSPACE-complete," *arXiv preprint arXiv:2605.22747*, 2026, accessed 2026-08-02.

[6] T. J. Schaefer, "On the complexity of some two-person perfect-information games," *Journal of Computer and System Sciences*, vol. 16, no. 2, pp. 185–225, 1978.

[7] G. Slatton, "Solving the board game Quoridor," https://grantslatton.com/solving-quoridor, May 2026, blog article and results table; accessed 2026-08-02.

[8] J. W. Romein and H. E. Bal, "Solving awari with parallel retrograde analysis," *IEEE Computer*, vol. 36, no. 10, pp. 26–33, 2003.

[9] P. Henderson, B. Arneson, and R. B. Hayward, "Solving 8x8 Hex," in *Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI-09)*, 2009, pp. 505–510.

[10] N. Wetzler, M. J. H. Heule, and W. A. Hunt, "DRAT-trim: Efficient checking and trimming using expressive clausal proofs," in *Theory and Applications of Satisfiability Testing (SAT 2014)*, ser. Lecture Notes in Computer Science, vol. 8561, 2014, pp. 422–429.

[11] M. J. H. Heule, O. Kullmann, and V. W. Marek, "Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer," in *Theory and Applications of Satisfiability Testing (SAT 2016)*, ser. Lecture Notes in Computer Science, vol. 9710, 2016, pp. 228–245.

[12] Y. K. Tan, M. J. H. Heule, and M. O. Myreen, "cake_lpr: Verified propagation redundancy checking in CakeML," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2021)*, ser. Lecture Notes in Computer Science, vol. 12652, 2021, pp. 223–241.

[13] J. Elffers, S. Gocht, C. McCreesh, and J. Nordström, "Justifying all differences using pseudo-Boolean reasoning," in *Proceedings of the 34th AAAI Conference on Artificial Intelligence (AAAI-20)*, vol. 34, 2020, pp. 1486–1494.

[14] L. Glendenning, "Mastering Quoridor," B.Sc. thesis, University of New Mexico, 2002.

[15] P. J. C. Mertens, "A Quoridor-playing agent," B.Sc. thesis, Maastricht University, 2006.

[16] V. Massagué Respall, J. A. Brown, and H. Aslam, "Monte Carlo tree search for Quoridor," in *Proceedings of the 19th International Conference on Intelligent Games and Simulation (GAME-ON 2018)*, 2018, pp. 5–9.

[17] T. Iwanaga, M. Sakamoto, T. Ito, and S. Ikeda, "Analysis of 5x5 board Quoridor," in *Proceedings of the International Conference on Artificial Life and Robotics (ICAROB 2022)*, vol. 27, 2022, pp. 398–401.

[18] ——, "Analysis of Quoridor by reusing the results of reduced version," in *Proceedings of the International Conference on Artificial Life and Robotics (ICAROB 2023)*, vol. 28, 2023, pp. 430–433.

[19] L. V. Allis, M. van der Meulen, and H. J. van den Herik, "Proof-number search," *Artificial Intelligence*, vol. 66, no. 1, pp. 91–124, 1994.

[20] G. Irving, J. Donkers, and J. Uiterwijk, "Solving Kalah," *ICGA Journal*, vol. 23, no. 3, pp. 139–147, 2000.

[21] E. C. D. van der Werf, H. J. van den Herik, and J. W. H. M. Uiterwijk, "Solving Go on small boards," *ICGA Journal*, vol. 26, no. 2, pp. 92–107, 2003.

[22] S. Even and R. E. Tarjan, "A combinatorial problem which is complete in polynomial space," *Journal of the ACM*, vol. 23, no. 4, pp. 710–719, 1976.

[23] S. Reisch, "Hex ist PSPACE-vollständig," *Acta Informatica*, vol. 15, no. 2, pp. 167–191, 1981, in German.

[24] M. Mhalla and F. Prost, "Gardner's minichess variant is solved," *ICGA Journal*, vol. 36, no. 4, pp. 215–221, 2013.

[25] V. Balabanov and J.-H. R. Jiang, "Unified QBF certification and its applications," *Formal Methods in System Design*, vol. 41, no. 1, pp. 45–65, 2012.

[26] A. Niemetz, M. Preiner, F. Lonsing, M. Seidl, and A. Biere, "Resolution-based certificate extraction for QBF (tool presentation)," in *Theory and Applications of Satisfiability Testing (SAT 2012)*, ser. Lecture Notes in Computer Science, vol. 7317. Springer, 2012, pp. 430–435.

[27] M. J. H. Heule, M. Seidl, and A. Biere, "Efficient extraction of Skolem functions from QRAT proofs," in *Formal Methods in Computer-Aided Design (FMCAD 2014)*. IEEE, 2014, pp. 107–114.

[28] B. Bogaerts, S. Gocht, C. McCreesh, and J. Nordström, "Certified dominance and symmetry breaking for combinatorial optimisation," *Journal of Artificial Intelligence Research*, vol. 77, pp. 1539–1589, 2023.

[29] A. L. Zobrist, "A new hashing method with application for game playing," Computer Sciences Department, University of Wisconsin, Tech. Rep. 88, 1970, reprinted in ICCA Journal, 13(2):69–73, 1990.

[30] D. E. Knuth and R. W. Moore, "An analysis of alpha-beta pruning," *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.

[31] A. Kishimoto and M. Müller, "A general solution to the graph history interaction problem," in *Proceedings of the 19th National Conference on Artificial Intelligence (AAAI-04)*, 2004, pp. 644–649.

[32] L. V. Allis, "Searching for solutions in games and artificial intelligence," Ph.D. dissertation, Maastricht University, 1994.

[33] A. Nagai, "Df-pn algorithm for searching AND/OR trees and its applications," Ph.D. dissertation, The University of Tokyo, 2002.